BIN702 - Série d'exercices sur la phylogénétique

Manuel Lafond

1 Méthodes par distance

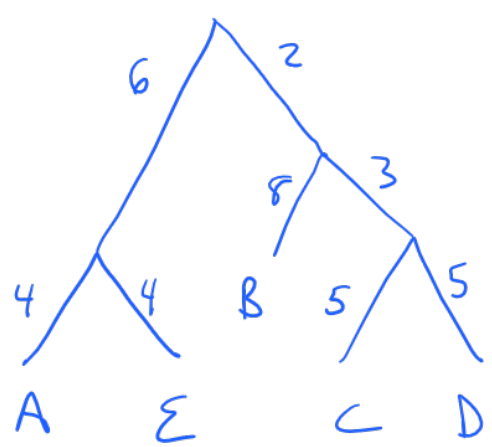
Exercice 1: Considérez la matrice de distances suivante :

	A	B	C	D	E
A	0	20	20	20	8
B		0	16	16	20
C			0	10	20
D				0	20
E					0

a. Est-ce que cette distance est ultra-métrique ? Si oui, reconstruisez l'arbre ultramétrique et sinon, dites pourquoi.

Solution.

Oui. L'arbre UPGMA est le suivant.



Une autre façon de montrer que les distances sont ultra-métriques est de regarder chaque triplet d'éléments et constater qu'on peut les nommer x, y, z tels que $D_{xy} = D_{xz} \geq D_{yz}$. Mais c'est plus long.

b. Est-ce que cette distance est additive ?

Solution.

Oui. Puisque la distance est ultra-métrique, elle est automatiquement additive. Ceci est parce qu'une distance ultra-métrique est comme une distance additive, mais avec la restriction que chaque distance racine-feuille soit identique.

Exercice 2: Montrez comment implémenter UPGMA en temps $O(n^2 \log n)$. Indice : il faut pouvoir mettre à jour une distance en temps $O(1)$ et utiliser une structure de données qui donne un accès rapide au minimum, comme un monceau.

Solution.

Si vous ne le saviez pas déjà, un *monceau* (ou heap en anglais) permet de stocker un ensemble de paires clé-valeur dans lequel on peut ajouter ou supprimer en temps $O(\log n)$, où n est le nombre d'éléments dans la structure. Et, plus important, UPGMA retourne en temps $O(1)$ la paire clé-valeur dont la clé est minimum. Les clés peuvent être répétées. Voir : `https://en.wikipedia.org/wiki/Heap_(data_structure)`.

On se rappelle que en cours, on a vue que lorsqu'on combine deux éléments u et v, on peut mettre à jour la distance entre $(u \cup v)$ et w en temps constant avec

$$D_{(u \cup v),w} = (|u| D_{u,w} + |v| D_{v,w})/(|u| + |v|)$$

Voir : `https://en.wikipedia.org/wiki/UPGMA`. On peut donc appliquer l'algorithme suivant.

```
fonction upgma(X, D)
    //X est l'ensemble de taxa, D la matrice de distances;
    M = Monceau();
    pour chaque paire d'éléments u, v de X faire
        //D_{u,v} est la clé, (u, v) est la valeur;
        M.insert(D_{u,v}, (u, v)) ;
    fin
    tant que |X| ≥ 2 faire
        done = False;
        tant que done = False faire
            (d, (u, v)) = M.extractMin() ;      // Retire un élément
             avec clé minimum et le retourne
            si u ∈ X et v ∈ X alors
                done = True;
            fin
        fin
        Joindre u à v dans l'arbre;
        Ajouter (u ∪ v) à X;
        Enlever u et v de X;
        pour chaque w ∈ X, où w ≠ (u ∪ v) faire
            D_{(u∪v),w} = (|u|D_{u,w} + |v|D_{v,w})/(|u| + |v|);
            M.insert(D_{(u∪v),w}, ((u ∪ v), w);
        fin
    fin
```

Justification courte.
Soit $n = |X|$. Pour un vétéran d'algorithmique, il n'est pas difficile de voir que le tout prend un temps $O(n^2 \log n)$, car la complexité est dominée par la deuxième boucle. Celle-ci est exécutée $O(n)$ fois et doit faire $O(n)$ insertions dans M, chacune en temps $O(\log n)$. La complexité est donc $O(n^2 \log n)$.

Justification longue.
Il est toutefois instructif d'entrer dans les détails de l'analyse. Notons que la première boucle prendra un temps $O(n^2 \log n)$, car il y a $O(n^2)$ paires d'éléments et chaque insertion prend $O(\log n)$.

La deuxième boucle "tant que $|X| \geq 2$" s'exécute $O(n)$ fois, car à chaque tour on combine deux éléments (et il y a n éléments au total).

La boucle "tant que $done = False$" est un peu plus subtile. Dans le pire des cas, on fera un $extractMin$ pour chaque élément qui sera inséré à un moment

ou à un autre dans M. Le nombre de $extractMin$ que l'on fera sera égal au nombre $O(n^2)$ d'insertions faites au départ, plus le nombre total d'insertions faites dans la toute dernière boucle. On fait $O(n)$ insertions par tout de boucle, donc au total, on fera $O(n^2)$ insertions dans cette dernière boucle. Le nombre de $extractMin$ que l'on fera, au total dans toute l'exécution de l'algorithme, sera $O(n^2 + n^2) = O(n^2)$. Puisque chaque $extractMin$ prend un temps $O(1)$, on passe un temps total de $O(n^2)$ dans les extarctions.

Finalement, à chaque tour de boucle, on joint deux éléments, on enlève de X et on ajoute à X, chaque opération se faisant en temps $O(n)$. Ensuite, encore à chaque tour de boucle, on fait $O(n)$ insertions dans M, chacune en temps $O(\log n)$. Le temps d'un tour de boucle est donc borné par $O(n + n \log n) = O(n \log n)$. Puisqu'il y a $O(n)$ tours de boucle, le temps passé dans la boucle principale est borné par $O(n \cdot n \log n) = O(n^2 \log n)$.

Au total, le temps de l'algorithme est

$$
\begin{aligned}
&O(\text{“temps de la 1re boucle”} + \text{“temps des extractMin”} + \text{“temps de la 2e boucle”}) \\
&= O(n^2 \log n + n^2 + n^2 \log n) \\
&= O(n^2 \log n)
\end{aligned}
$$

Exercice 3: Soit D une distance ultra-métrique et soit T l'arbre reconstruit avec UPGMA. Dans la version présentée en classe, nous n'avons pas montré comment étiqueter les arêtes de façon à respecter les distances, et de façon à ce que chaque distance feuille-racine soit la même.
Donnez un algorithme qui trouve un étiquetage de T satisfaisant les conditions d'ultramétricité.

Solution.
On observe que si T est un arbre ultra-métrique, pour n'importe quel sommet x, la distance de x à une feuille descendante de x est toujours la même. Donc la propriété d'ultra-métricité est vraie pour tous les sous-arbres.
On peut donc parcourir T de bas en haut de façon à ce que, pour chaque noeud x, le sous-arbre T_x soit ultra-métrique. Après avoir traité T_x, on stocke

la distance de x à ses feuilles et on réutilisera l'information plus haut.

```
fonction ultraMetricDist(T, D)
    R = [ ] ; // R[x] entreposera la dist de x à ses feuilles.
    dist_branche = [ ];                    // Ce qu'on va retourner.
    pour chaque noeud x dans un parcours post-ordre faire
        si x est une feuille alors
            R[x] = 0;
        sinon
            Soient x_1, x_2 les enfants de x;
            Soit a une feuille sous x_1;
            Soit b une feuille sous x_2;
            d = D_{a,b} ;    // d est le même pour tout choix de a, b
            //Pour rendre T_x ultra-métrique, on envoie d/2 de chaque ;
            //côté en ajustant les branches selon R[x_1] et R[x_2];
            dist_branche[x, x_1] = d/2 − R[x_1];
            dist_branche[x, x_2] = d/2 − R[x_2];
            R[x] = d/2;
        fin
    fin
    return dist_branche;
```

Exercice 4: Démontrez la condition de trois points, c'est-à-dire qu'une distance D est ultra-métrique si et seulement si pour chaque trois taxa, on peut les nommer x, y, z de façon à ce que $D_{xy} = D_{xz} \geq D_{yz}$.

Solution.

Il n'y aura pas de question de ce style à l'examen. Si c'est tout ce qui vous inéresse, passez cette preuve et allez au prochain numéro ! Sinon, voici.

On dénote $lca_T(a, b)$ le dernier ancêtre commun de a et b dans un arbre T. On écrit aussi $dist_T(a, b)$ pour dénoter la somme des poids des branches sur le chemin de a à b dans T. Rappelons que pour démontrer un "si et seulement si", il faut montrer les deux directions de l'implication.

($\Rightarrow$) : supposons que D est ultra-métrique. Soit T un arbre satisfaisant les conditions d'ultra-métricité. Soient x, y et z. Supposons que x, y, z soient choisis de façons à ce que $lca_T(x, y)$ soit un descendant de $lca_T(x, z)$ (sinon, on permute les rôles de x, y, z). On note que $lca_T(x, z) = lca_T(y, z)$. Dénotons $p = lca_T(x, y)$ et $q = lca_T(x, z)$. On peut observer que $dist_T(x, p) = dist_T(y, p)$, parce que T est ultra-métrique (s i ce n'était pas le cas, impossible que $dist(x, racine) = dist(y, racine)$). De la même façon, $dist_T(x, q) = dist_T(y, q) = dist_T(z, q)$. Ceci

veut dire que

$$dist_T(x,z) = dist_T(y,z) = 2 \cdot dist_T(x,q)$$
$$dist_T(x,y) = 2 \cdot dist_T(x,p)$$

Il est évident que $dist_T(x,p) \leq dist_T(x,q)$, et donc on a $dist_T(x,z) = dist_T(y,z) \geq dist_T(x,y)$, la propriété recherchée.

($\Leftarrow$) : on doit maintenant prouver l'autre direction. C'est-à-dire, si $D_{xy} = D_{xz} \geq D_{yz}$, alors il est possible de reconstruire un arbre ultra-métrique pour D. On peut le démontrer par induction sur n, le nombre de taxa. Si $n = 2$, D est trivialement ultra-métrique. On suppose donc par induction que si une distance D' a $n-1$ taxa et satisfait la condition des 3 points, alors on peut construire un arbre T. Prenons D avec ses n taxa. Soient a, b les taxa qui minimisent $D_{a,b}$. Soit D' la distance obtenue de D en retirant a. Alors on peut appliquer l'induction sur D' car elle satisfait toujours la condition des 3 points. Soit T' un arbre pour D. Alors, b est une feuille de T' et la longueur de la branche de b à son parent est au moins $D_{a,b}/2$ (sinon, $D_{a,b}$ ne serait pas minimum - si vous ne voyez pas pourquoi, demandez-moi !). Soit p le parent de b sur T' et soit ℓ la longueur de cette branche. On peut greffer a sur la branche (p, b), c'est-à-dire créer un nouveau noeud q entre p et b et ajouter a comme enfant de q. On attribue une longueur de $D_{a,b}/2$ à (q, b) et à (q, a), et une longueur de $\ell - D_{a,b}/2$ à (p, q). Ceci ne change pas les distances entre b et un autre noeud de T'. De plus, la distances entre a et un noeud x dans ce T' modifié est la même pour a et pour b. Vous pouvez vérifier que $D_{a,b}$ étant minimum, la distance D devait satisfaire cette propriété. On déduit que T' avec a ajouté contient bien les distances de D.

Exercice 5: Quelle est la complexité de l'algorithme Neighbor-Joining ? On peut argumenter qu'il s'implémente en temps $O(n^3)$, où n est le nombre de taxa qui étaient présent au départ.

Solution.

Rappelons l'algorithme de Neighbor-Joining tel que présenté en classe.

fonction *NJ(D)*

Soit T un arbre étoile avec une feuille par taxa;
$n =$ nombre de taxa;
tant que *le centre de T a au moins 4 voisins* **faire**
Choisir i, j qui minimisent $(n-2)D_{i,j} - \sum_{k=1}^{n}(D_{i,k} - D_{j,k})$;
Joindre i et j dans T, appeler le nouveau noeud créé u;
$n = n - 1$;
pour chaque $k \neq i, j, u$ **faire**
$D_{u,k} = 1/2(D_{i,k} + D_{j,k} - D_{i,j})$;
fin
fin
return T;

On fait $O(n)$ tours de boucle, et la recherche du i, j minimum prend un temps $O(n^3)$. Ceci donne une complexité $O(n^4)$. On peut faire mieux en maintenant

un tableau Q des valeurs à optimiser.

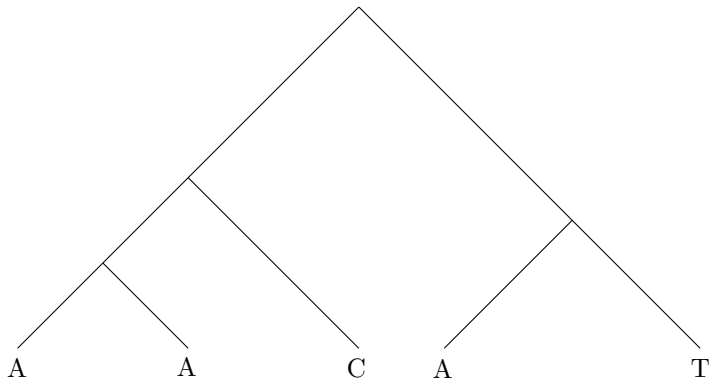
```
fonction NJ(D)
    Soit T un arbre étoile avec une feuille par taxa;
    n = nombre de taxa;
    Soit Q un tableau 2D;
    pour chaque i, j faire
        Q_{i,j} = (n-2)D_{i,j} - Σ_{k=1}^{n}(D_{i,k} - D_{j,k});
    fin
    tant que le centre de T a au moins 4 voisins faire
        Choisir i, j qui minimisent Q_{i,j} ;
        Joindre i et j dans T, appeler le nouveau noeud créé u;
        n = n - 1;
        pour chaque k ≠ i, j, u faire
            D_{u,k} = D_{k,u} = 1/2(D_{i,k} + D_{j,k} - D_{i,j});
        fin
        //Mettre à jour Q_{u,k} pour tout k;
        pour chaque k ≠ i, j, u faire
            Q_{u,k} = (n-2)D_{u,k} - Σ_{l=1}^{n}(D_{u,l} - D_{k,l});
        fin
        //Mettre à jour Q_{a,b} pour les autres paires;
        //La différence : on ne compte plus -D_{a,i}, -D_{a,j}, -D_{b,i}, -D_{b,j};
        //la valeur de n a changé, et on compte -D_{a,u}, -D_{b,u};
        pour chaque a, b ≠ i, j, k faire
            Q_{a,b} = Q_{a,b} - D_{a,b} //la valeur de n a changé;
            Q_{a,b} = Q_{a,b} + D_{a,i} + D_{a,j} + D_{b,i} + D_{b,j} //car i, j supprimés;
            Q_{a,b} = Q_{a,b} - D_{a,u} - D_{b,u} //car on a ajouté u;
        fin
    fin
    return T;
```

La boucle qui initialise Q prend un temps $O(n^3)$. On peut ensuite vérifier que chaque boucle imbriquée prend u ntemps $O(n^2)$, donc la boucle pricipale prend un temps $O(n^3)$.

2 Méthodes par caractères et par vraisemblance

Exercice 6: Soit l'arbre suivant :

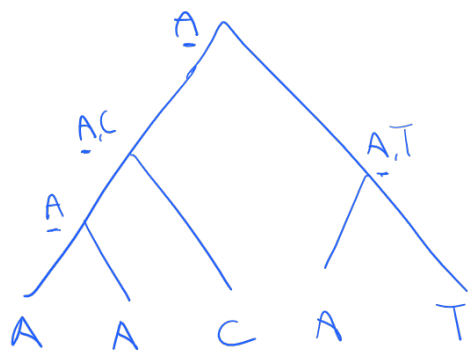
a. Exécutez l'algorithme de Fitch permettant de minimiser les changements sur cet arbre. Donnez les caractères candidats aux ancêtres ainsi qu'un



étiquetage optimal.

Solution.

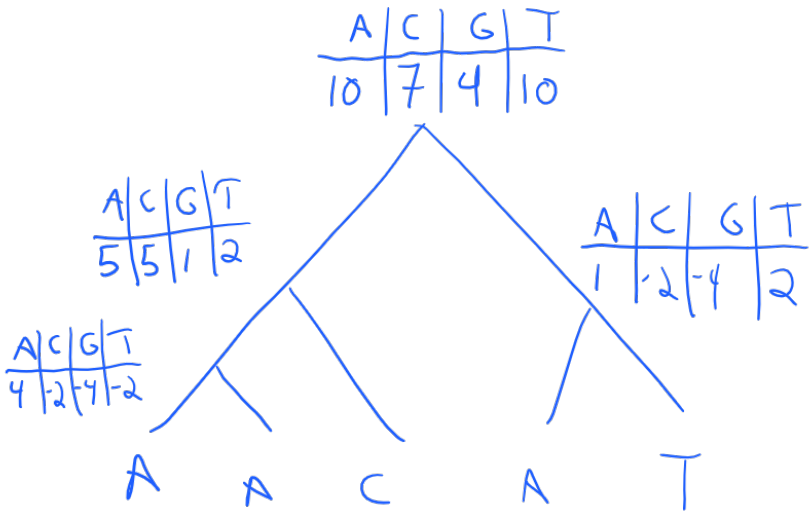
L'étiquetage optimal place un A partout.



b. Exécutez l'algorithme de Sankof-Rousseau permettant de maximiser le score des branches sur cet arbre. Utilisez comme matrice de score

	A	C	G	T
A	2	-1	-2	-1
C	-1	2	-1	-1
G	-2	-1	1	-2
T	-1	-1	-2	3

Solution. (en supposant l'absence d'erreur de calcul, voir page suivante)



Une assignation optimale serait de placer un A partout.

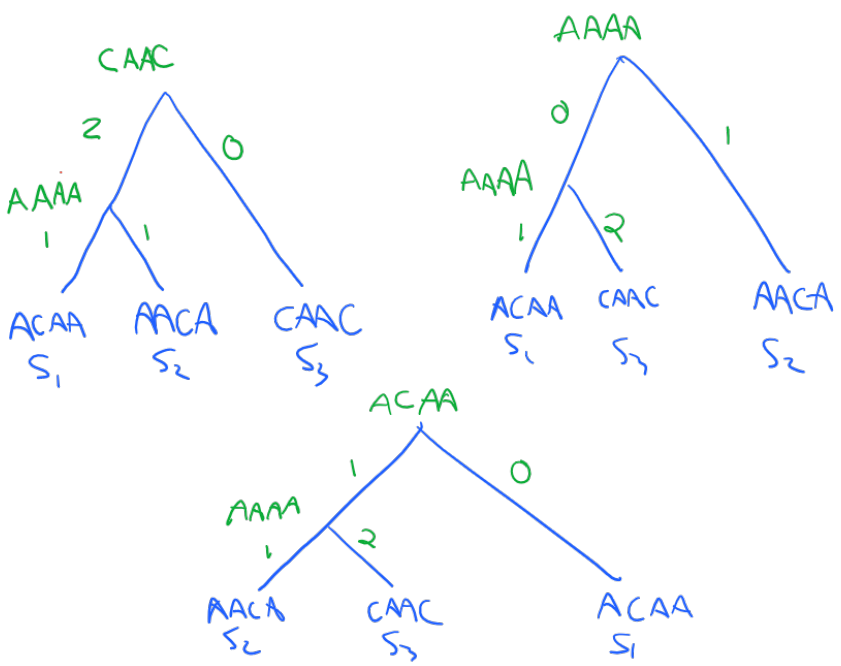
Exercice 7: Supposez qu'il n'existe que les caractères A et C. Soient les séquences

$$S_1 = ACAA$$
$$S_2 = AACA$$
$$S_3 = CAAC$$

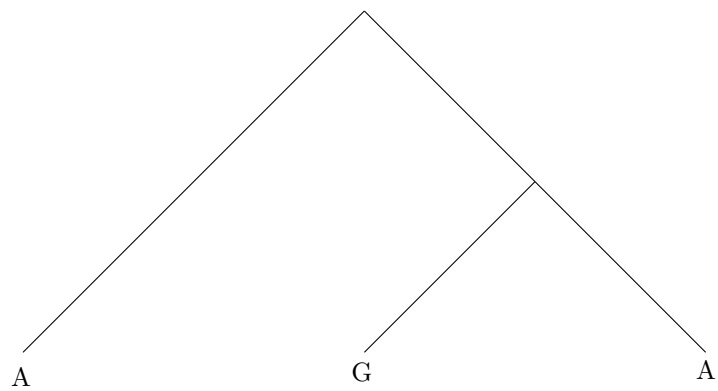
Quel est l'arbre le plus parcimonieux, selon la distance Hamming ? C'est-à-dire, quel arbre minimise la somme des nombres de changements sur les branches pour l'ensemble des positions ?

Solution.

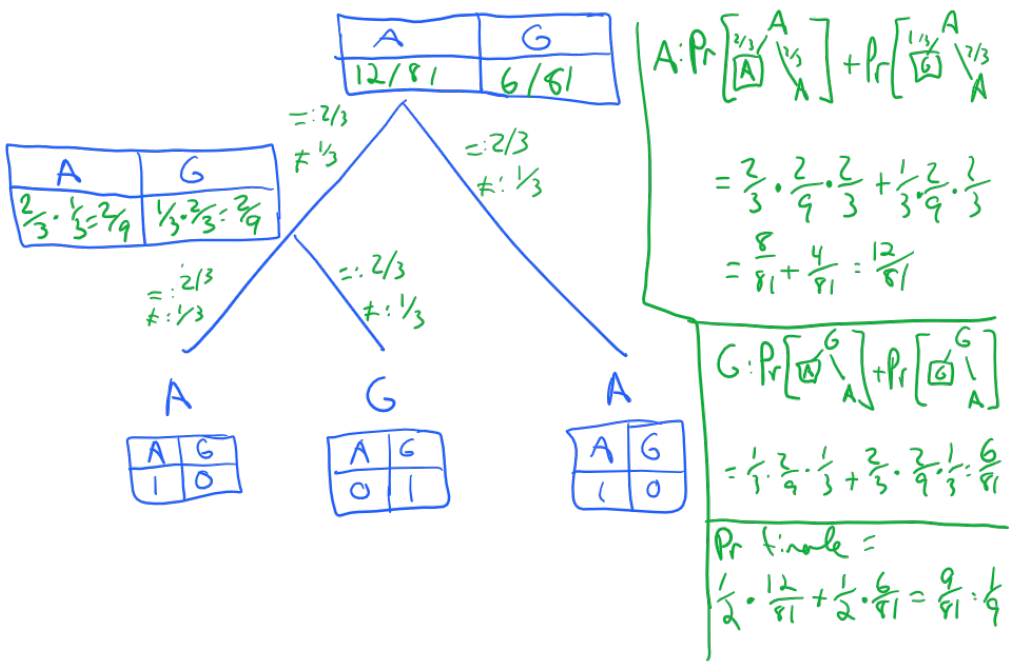
Il n'y a que 3 arbres, donc on peut se permettre de tous les évaluer. Sur chaque arbre, j'ai manuellement exécuté l'algorithme de Fitch sur chaque caractère. Le figure ci-bas donne les solutions optimales sur chaque arbre. Il s'avère qu'ils sont tous optimaux et mènent chacun à un minimum de 4 changements.



Exercice 8: Supposez qu'il n'existe que deux caractères possibles A et G. Soit l'arbre suivant :



Considérez que sur chaque branche, la probabilité qu'un caractère reste le même est 2/3 et la probabilité de changer est 1/3. Quel est le score de vraisemblance de cet arbre ?



Solution.
J'utilise la récurrence présentée en classe. La figure ci-dessus illustre les grandes étapes. En bas à droite, la vraisemblance finale de l'arbre.

Exercice 9: Dans l'algorithme de Sankoff-Rousseau, nous avons montré comment calculer, pour chaque noeud x et caractère a, une valeur représentant le

score maximum de T_x en forçant $x = a$. Nous n'avons pas montré comment reconstruire les états ancestraux.
Montrez comment, une fois que toutes les valeurs sont calculées, comment inférer un étiquetage des noeuds par un caractère de façon à maximiser la somme des scores des arêtes.

Solution.
Rappelons que nous utilisons la récurrence

$$V[x, a] = \max_{b \in \Sigma}(V[x_1, b] + M[a, b]) + \max_{b \in \Sigma}(V[x_2, b] + M[a, b])$$

Pour reconstruire un état à chaque noeud, on commence par la racine r. On choisit le caractère a qui maximise $V[r, a]$. Ensuite, en parcourant de haut en bas, pour chaque noeud x, on base le caractère en x par rapport au caractère choisi à son parent p. Pour ce faire, on choisit le caractère en x qui a donné lieu à l'optimal choisi pour p.

fonction *backtrackSankoffRousseau(T, V)*
- $C = \{\}$ //contient le caractère affecté à chaque noeud;
- Soit r la racine de T;
- $C[r] = \arg\max_{a \in \Sigma} V[r, a]$;
- //rappel : dans un parcours pré-ordre, un noeud est visité avant ses enfants;
- **pour chaque** *noeud x dans un parcours pré-ordre, excluant r* **faire**
  - **si** *x est une feuille* **alors**
    - $C[x] =$ le caractère assigné à x;
  - **sinon**
    - Soit p le parent de x;
    - $a = C[p]$;
    - Choisir $b \in \Sigma$ qui maximise $V[x, b] + M[a, b]$;
    - $C[x] = b$;
  - **fin**
- **fin**
- return C;

Exercice 10: Dans l'algorithme de large parcimonie, on doit itérer sur tous les arbres binaires avec n feuilles étiqutées. Mais combien d'arbres doit-on énumérer ?
Montrez que le nombre d'arbres binaires enracinés avec n feuilles étiquetées par une séquence est $(2n-3)(2n-5)(2n-7)\ldots \cdot 3 \cdot 1$.

Solution.
Notons qu'un arbre avec n feuilles a $2n-2$ arêtes. Soit $t(n)$ le nombre d'arbres enracinés sur n feuilles étiquetées. Il est facile de voir que $t(2) = 1$. Pour $n = 3$, on peut prendre l'unique arbre T avec 2 feuilles, et ajouter la 3ème feuille sur une des 2 branches existantes, ou bien créer une nouvelle racine et lui donner comme enfants T et la nouvelle feuille. Il y a donc $3 \cdot 1$ façons d'avoir un arbre à 3 feuilles.
Généralisons à un n arbitraire. On a $t(n-1) = (2(n-1)-3)(2(n-1)-5)\ldots \cdot 3 \cdot 1 = (2n-5)(2n-7)\ldots \cdot 3 \cdot 1$ arbres avec $n-1$ feuilles. Chacun de ces arbres a $2(n-1)-2 = 2n-4$ branches. Pour avoir n feuilles, il faut prendre un de ces arbres et greffer la nouvelle feuille sur une de ces branches, ou l'ajouter au-dessus de la racine. Ceci donne $2n-4+1 = 2n-3$ façons, et ce pour chacun des $t(n-1)$ arbres. Le nombre de façons de créer un arbre avec n feuilles est donc $(2n-3)t(n-1) = (2n-3)(2n-5)(2n-7)\ldots \cdot 3 \cdot 1$.